

PROJECT REPORT

Smart Expense Tracker

Project Title	Smart Expense Tracker
---------------	-----------------------

Submitted by	Harshita Gupta
--------------	----------------

Registration number	26BCE10334
---------------------	------------

Faculty	Dr. Pradeep Kumar Mishra
---------	--------------------------

Submission date	30 September 2026
-----------------	-------------------

1. Introduction

Students often make frequent small purchases for food, travel, stationery, and other needs. Without a record, it is difficult to understand spending habits. Smart Expense Tracker is a simple Python application with a Streamlit browser interface that records these purchases and presents a clear monthly summary. It is designed to apply fundamental Python concepts such as functions, classes, lists, conditions, loops, file/database handling, and input validation to a real-world problem.

2. Problem Statement

Students need a straightforward method to record personal expenses, group them into categories, and compare category spending with a monthly plan. Manual notes are easily forgotten and do not automatically calculate totals. The project solves this problem using a local Python program and SQLite database.

3. Functional Requirements

ID	Requirement
FR1	The system shall display a menu so the user can choose an action.
FR2	The user shall be able to add and list expense categories.
FR3	The user shall be able to add an expense with amount, date, category, and description.
FR4	The user shall be able to view all expenses or filter them by month.
FR5	The user shall be able to edit or delete an expense by ID.
FR6	The system shall calculate total monthly spending and category-wise monthly spending.
FR7	The user shall be able to set a monthly category budget and view its status.

The three major functional modules are category management, expense management, and reporting/budget monitoring.

5. Non-functional Requirements

Area	Requirement and implementation
Usability	A numbered menu, short prompts, readable tables, and clear success/error messages make the program simple to use.
Reliability	SQLite transactions and database constraints preserve valid local records.
Performance	Queries work quickly for a typical student's small personal expense list.
Maintainability	Database, validation, business logic, display, and menu code are separated into small modules.
Error handling	Invalid amounts, dates, months, blank text, duplicate categories, and unknown IDs produce friendly messages instead of ending the program.
Resource efficiency	The project uses Python's standard library and one lightweight local SQLite file.

6. System Architecture

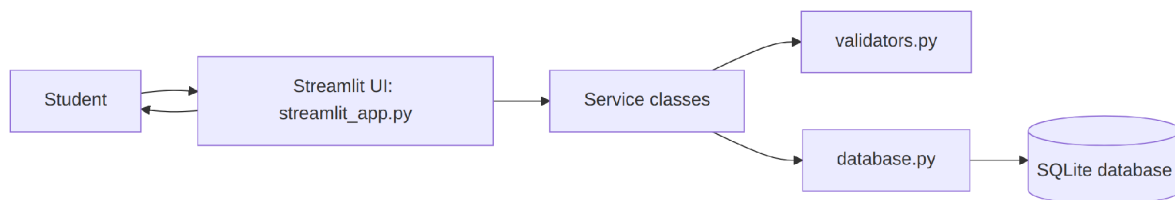


Figure 1: System architecture

`streamlit_app.py` receives the user's input through forms and buttons. It calls service classes, which validate data and read/write SQLite through the Database class. Streamlit displays tables, messages, and the monthly chart.

7. Design Diagrams

Use Case Diagram

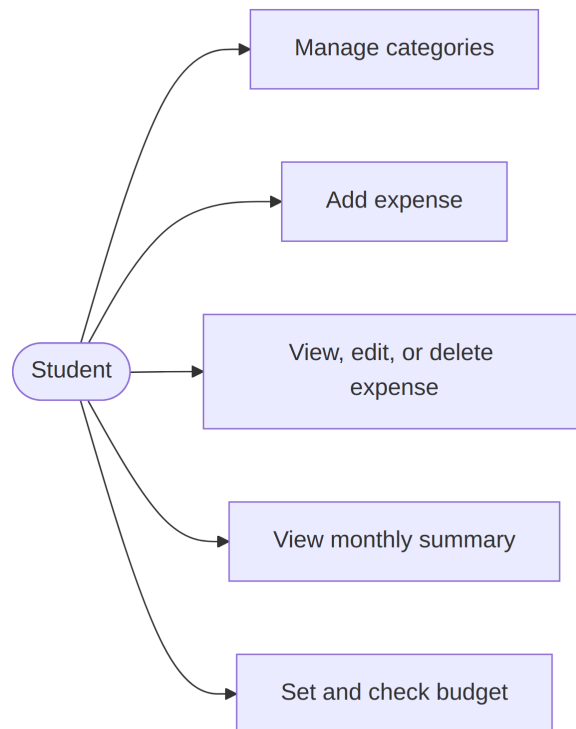


Figure 2: Use case diagram

Workflow Diagram

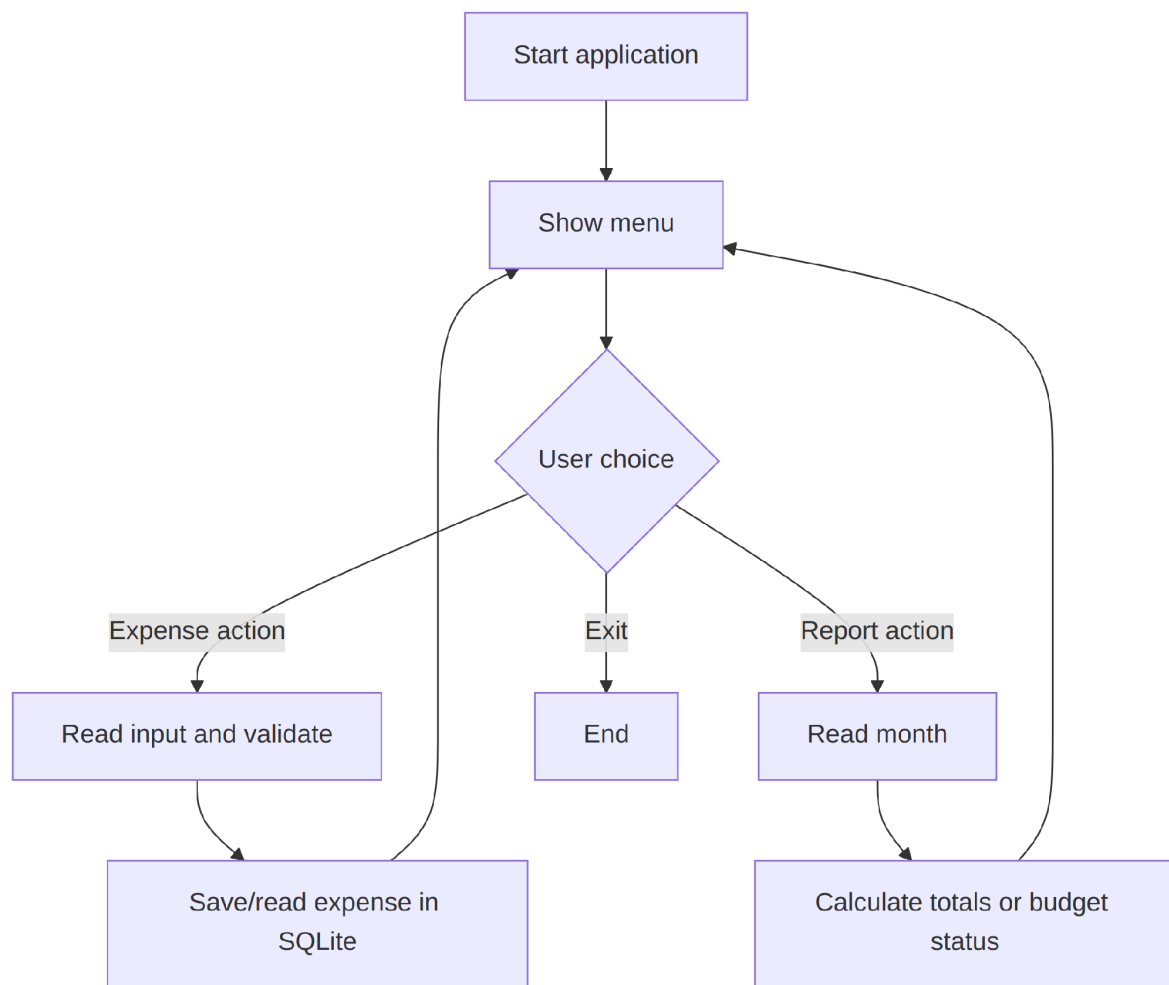


Figure 3: Workflow diagram

Sequence Diagram: Add Expense

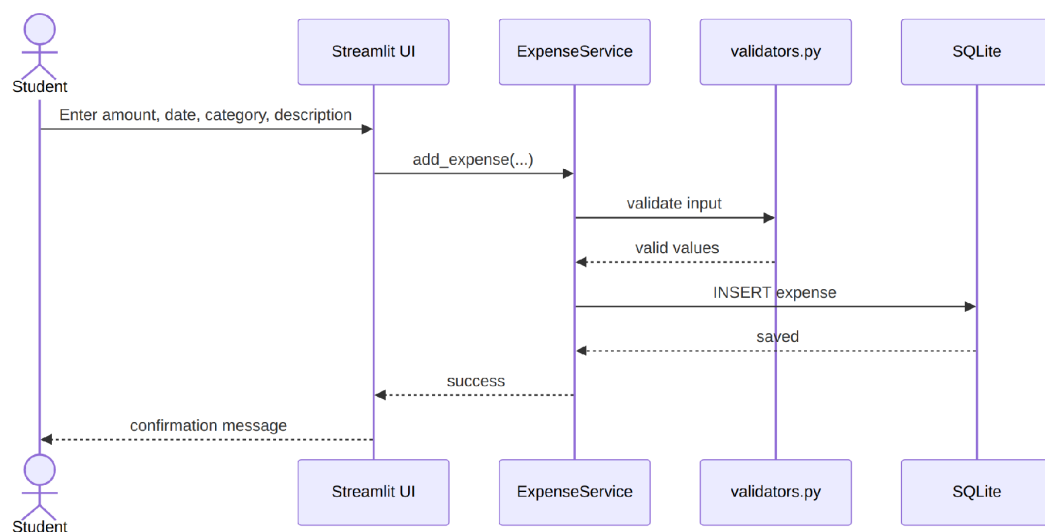


Figure 4: Sequence diagram for adding an expense

Class/Component Diagram

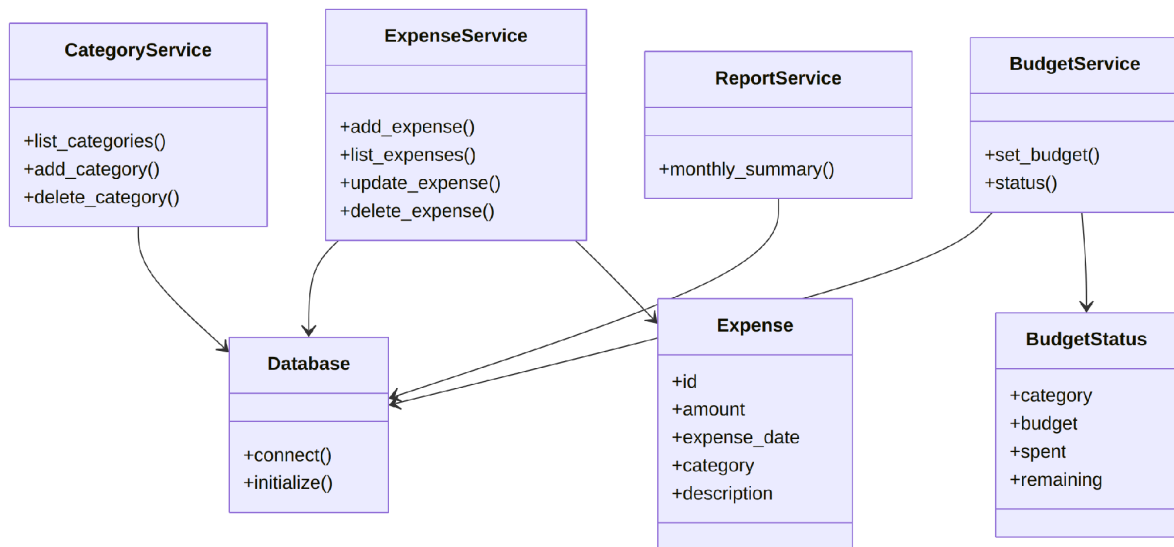


Figure 5: Class/component diagram

ER Diagram and Schema Design

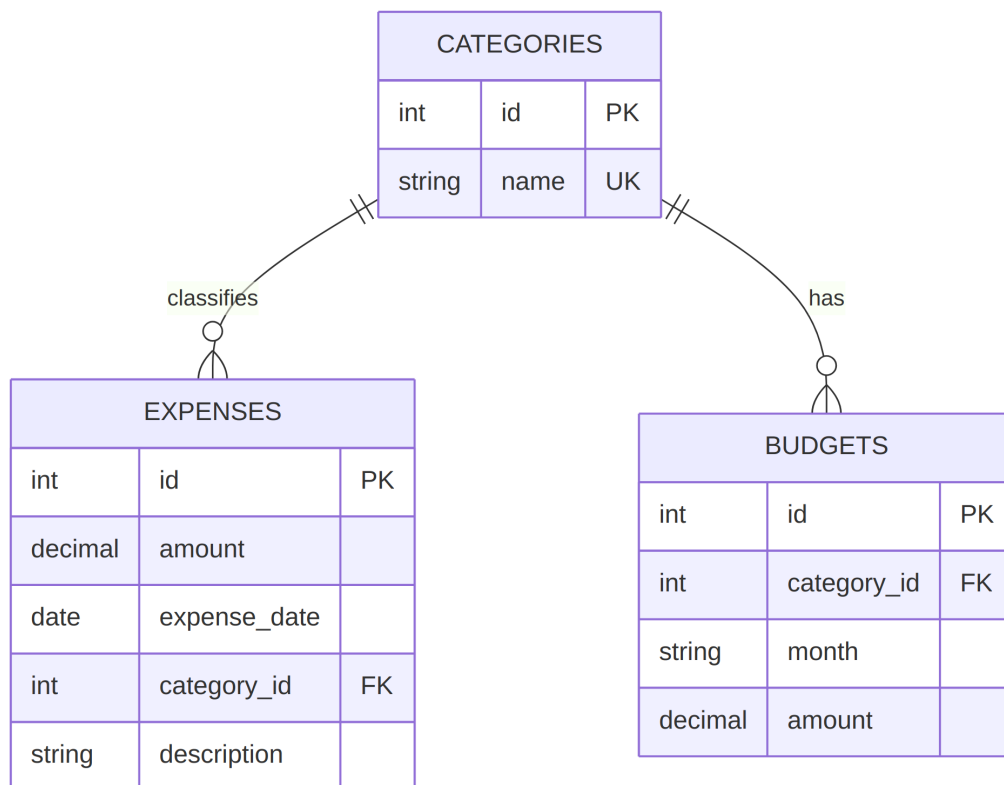


Figure 6: Entity-relationship diagram

categories(id, name) stores category names. expenses(id, amount, expense_date, category_id, description, created_at) stores each expense. budgets(id, category_id,

month, amount) stores one planned amount per category and month; (category_id, month) is unique.

8. Design Decisions and Rationale

- Python was selected because it matches the project requirement and is currently being taught as part of the course *Python Essentials* by Dr. Lakshmi D.
- Streamlit was selected because it creates a clear browser interface with simple Python code, without requiring complex web development.
- SQLite was selected because it is included with Python, requires no server, and still demonstrates structured storage and SQL.
- Separate service and validation modules make the code easier to read, test, and explain without using an unnecessarily complex architecture.

9. Implementation Details

File	Responsibility
streamlit_app.py	Streamlit pages, forms, tables, and user interaction.
database.py	Creates tables, starter categories, and SQLite connections.
services.py	Category, expense, budget, and report operations.
validators.py	Checks amounts, dates, months, and text fields.
models.py	Defines simple Expense and BudgetStatus data classes.
tests/	Automated validator and service tests.

The database is created automatically at `data/expense_tracker.db` when `python -m streamlit run streamlit_app.py` is run. The initial categories are Food, Travel, Shopping, Bills, and Other.

10. Screenshots / Results

Deploy

Smart Expense Tracker

A simple personal expense tracker for students.

Expenses

Add an expense

Amount (Rs.)

Example: 125.50

Date

2026 / 09 / 28

Category

Bills (ID: 4)

Description

Example: Lunch

Add expense

Deploy

Monthly Report

A simple personal expense tracker for students.

Month (YYYY-MM)

2026-09

Show monthly report

Total spending

Rs. 2575.00

Category	Total (Rs.)
Shopping	2500
Food	75

Deploy

Monthly Report

A simple personal expense tracker for students.

Month (YYYY-MM)

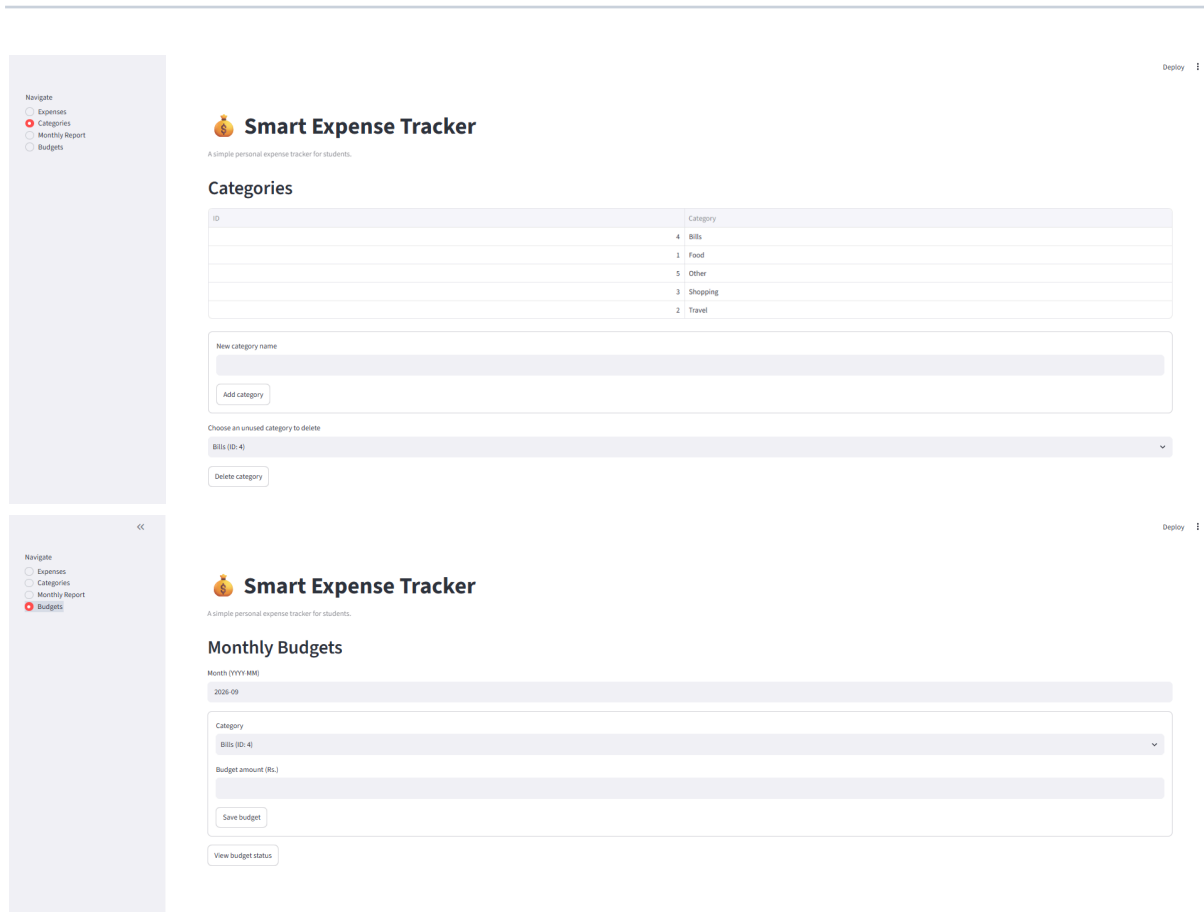
2026-09

Show monthly report

Total spending

Rs. 2575.00

Category	Total (Rs.)
Shopping	2500
Food	75



11. Testing Approach

Automated tests use Python's unittest module and a temporary SQLite database. Run them with:

```
python -m unittest discover -s tests -v
```

Test area	Expected result
Valid input	Amount, date, month, and text are accepted and normalized.
Invalid input	Zero/negative/non-numeric amount, invalid date/month, and blank text raise a clear validation error.
Expense CRUD	An expense can be added, read, updated, and deleted.
Reporting	Monthly totals include stored expenses in the selected month.
Budget	A category with spending above its saved budget has a negative remaining value.

12. Challenges Faced

- Designing a storage structure that remains understandable while linking expenses and budgets to categories.
- Handling invalid form input without stopping the entire application.
- Presenting database records in clear browser tables.

13. Learnings and Key Takeaways

- A real problem can be divided into input, processing, storage, and output stages.
- Classes and modules improve readability when each has one clear responsibility.
- Input validation and tests are important even in a small console application.
- SQLite provides a practical introduction to relational databases, tables, keys, and SQL queries.
- Git commits document the progression of a project.

14. Future Enhancements

- Export a monthly report to CSV.
- Add user accounts if the project is later converted to a multi-user application.
- Show simple charts after learning an appropriate visualization library.

15. References

- [1] Python Software Foundation, *Python 3 Documentation*: <https://docs.python.org/3/>
- [2] SQLite, *SQLite Documentation*: <https://www.sqlite.org/docs.html>
- [3] Assignment document, *VITyarthi — Build Your Own Project General Project Instructions & Submission Guidelines*.